



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №9
«КЛАССИФИКАЦИЯ
ДАННЫХ»**

по дисциплине «Языки программирования для статистической обработки
данных»

Студент группы

ИНБО-20-23. Боргачев Т.М.

(подпись)

Преподаватель

Трушин СМ

(подпись)

Москва 2025 г.

ВВЕДЕНИЕ

Цель практики: освоить построение моделей классификации (логистическая регрессия и KNN) с использованием Python, R, а также интерпретацию результатов классификации через отчёты.

Задачи практики:

1. Построить модель логистической регрессии:
 - Python: использование библиотеки `sklearn` и `statsmodels`.
 - R: функция `glm()` с семейством `binomial`.
2. Построить модель К-ближайших соседей (KNN):
 - Python: библиотека `sklearn` (`KNeighborsClassifier`).
 - R: использование пакета `class` или `caret`.
3. Выполнить прогноз на тестовых данных:
 - Python и R: метрики оценки точности (`accuracy`, `precision`, `recall`).
4. Интерпретировать результаты моделей:
 - Выявить важные переменные в логистической регрессии.
 - Оценить точность моделей с помощью ROC-кривой, матрицы ошибок.
 - Сравнить результаты между Python, R

РЕЗУЛЬТАТЫ РАБОТЫ

Подготовка данных

Код для подготовки данных на Python представлен в листинге 1.

Листинг 1 – Предобработка данных в Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# Загрузка данных
df = pd.read_csv('features_3_sec.csv')

# Удаляем неинформативные признаки
X = df.drop(['filename', 'label'], axis=1)
y = df['label'].map({'blues': 0, 'classical': 1, 'country': 2, 'disco': 3, 'hiphop': 4, 'jazz': 5, 'metal': 6, 'pop': 7, 'reggae': 8, 'rock': 9})

# Разделение на train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Масштабирование признаков (критично для KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Проверка дисперсии
variances = np.var(X_train_scaled, axis=0)
zero_var_features = np.where(variances == 0)[0]

if len(zero_var_features) > 0:
    print(f"Удаляем признаки с нулевой дисперсией: {zero_var_features}")
    X_train_scaled = np.delete(X_train_scaled, zero_var_features, axis=1)
    X_test_scaled = np.delete(X_test_scaled, zero_var_features, axis=1)
```

Код для подготовки данных на R представлен в листинге 2.

Листинг 2 – Предобработка данных в R

```
library(nnet)
library(class)
library(caret)
library(pROC)
library(ggplot2)

# 1. Загрузка данных
df <- read.csv("features_3_sec.csv", stringsAsFactors = FALSE)

# Предобработка данных
X <- df[, !(names(df) %in% c("filename", "label"))]
y <- as.factor(df$label)

# Разделение на train/test (70/30)
set.seed(42)
trainIndex <- createDataPartition(y, p = 0.7, list = FALSE)
X_train <- X[trainIndex, ]
X_test <- X[-trainIndex, ]
y_train <- y[trainIndex]
y_test <- y[-trainIndex]

# Удаление признаков с нулевой дисперсией и корреляцией
preProc <- preProcess(X_train, method = c("zv", "corr"))
X_train_processed <- predict(preProc, X_train)
X_test_processed <- predict(preProc, X_test)

# Масштабирование
X_train_scaled <- scale(X_train_processed)
X_test_scaled <- scale(X_test_processed)
```

Построение модели логистической регрессии

Код для построения модели логистической регрессии на Python представлен в листинге 3.

Листинг 3 – Модель логистической регрессии в Python

```
from sklearn.linear_model import LogisticRegression
import statsmodels.api as sm

# Стандартная модель
logreg = LogisticRegression(max_iter=1000, multi_class='ovr')
logreg.fit(X_train_scaled, y_train)
y_pred_logreg = logreg.predict(X_test_scaled)

# Статистическая модель для многоклассовой классификации
X_train_sm = sm.add_constant(X_train_scaled)
model_sm = sm.MNLogit(y_train, X_train_sm).fit()
print(model_sm.summary())
```

Код для построения модели логистической регрессии на R представлен в листинге

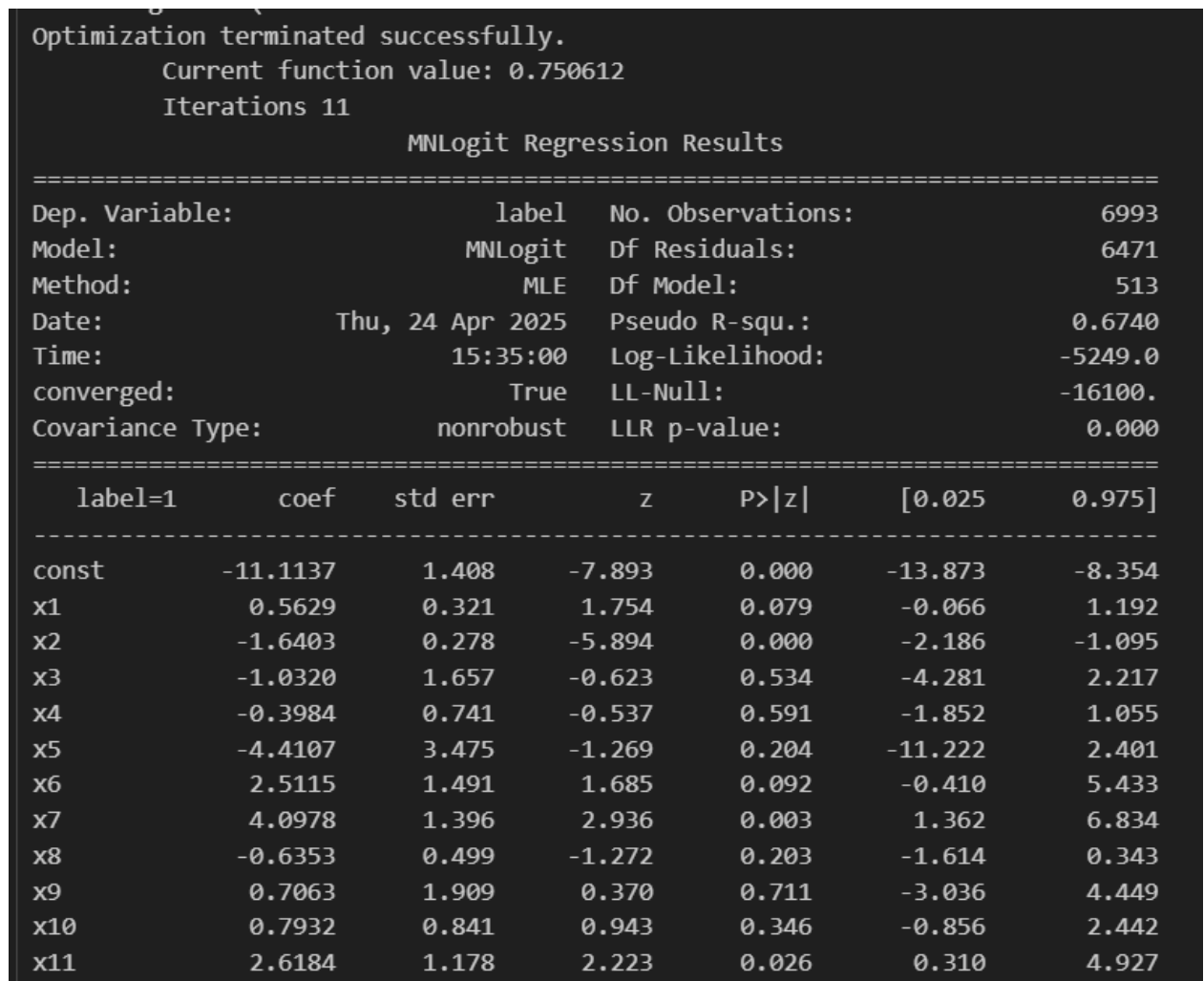
4.

Листинг 4 – Модель логистической регрессии в R

```
# 2. Логистическая регрессия (многоклассовая)
model_log <- multinom(y_train ~ ., data = as.data.frame(X_train_scaled))
summary(model_log)

# Предсказания
y_pred_logreg <- predict(model_log, newdata = as.data.frame(X_test_scaled))
y_pred_prob_logreg <- predict(model_log, newdata = as.data.frame(X_test_scaled),
type = "probs")
```

Результаты работы модели на Python представлены на рис. 1.



Optimization terminated successfully.
Current function value: 0.750612
Iterations 11

MNLogit Regression Results

Dep. Variable:	label	No. Observations:	6993
Model:	MNLogit	Df Residuals:	6471
Method:	MLE	Df Model:	513
Date:	Thu, 24 Apr 2025	Pseudo R-squ.:	0.6740
Time:	15:35:00	Log-Likelihood:	-5249.0
converged:	True	LL-Null:	-16100.
Covariance Type:	nonrobust	LLR p-value:	0.000

	label=1	coef	std err	z	P> z	[0.025	0.975]
const		-11.1137	1.408	-7.893	0.000	-13.873	-8.354
x1		0.5629	0.321	1.754	0.079	-0.066	1.192
x2		-1.6403	0.278	-5.894	0.000	-2.186	-1.095
x3		-1.0320	1.657	-0.623	0.534	-4.281	2.217
x4		-0.3984	0.741	-0.537	0.591	-1.852	1.055
x5		-4.4107	3.475	-1.269	0.204	-11.222	2.401
x6		2.5115	1.491	1.685	0.092	-0.410	5.433
x7		4.0978	1.396	2.936	0.003	1.362	6.834
x8		-0.6353	0.499	-1.272	0.203	-1.614	0.343
x9		0.7063	1.909	0.370	0.711	-3.036	4.449
x10		0.7932	0.841	0.943	0.346	-0.856	2.442
x11		2.6184	1.178	2.223	0.026	0.310	4.927

Рисунок 1 – Summary модели в Python

Результаты работы модели на R представлены на рис. 2.

```
Call:
multinom(formula = y_train ~ ., data = as.data.frame(X_train_scaled))

Coefficients:
              (Intercept) chroma_stft_mean chroma_stft_var rms_mean rms_var
classical    -2.3995401      0.15179469    -1.4800756  -4.4935300  -0.11218092
country       1.4507728     -0.46762572    -0.9985473  -6.2165469  -2.02178555
disco         1.4665555      0.58187777    -0.5661118  -3.5518504  -0.77286985
hiphop        1.3587601      2.05367756    -0.2847023   0.2605101  -0.24617786
jazz          -0.3422503     -0.47168886    -0.5794993  -0.7578223  -0.09354141
metal        -2.8682246      1.63655612    -0.2977774  -3.5037633  -2.40607342
pop           -0.8583275     -0.03997558    -1.1786543  -2.1473491  -0.71273814
reggae        1.7278324      0.68049034    -0.6270870  -1.4662094  -0.83838367
rock          1.3009566      0.27196899    -0.4243809  -2.0355296  -1.86443000
              spectral_centroid_var spectral_bandwidth_var rolloff_mean rolloff_var
classical      0.8451191          -0.70276484      0.7387988   1.08545199
country        1.9832157           0.28172608      0.3113479  -0.09249624
disco          1.3724755           0.71706285      2.8642030   0.48896338
hiphop         1.4496563           0.04465914      3.7037010   0.90810456
jazz           0.2684727          -0.73874998      3.1012929   0.92386400
metal          1.5986349           1.66092579      4.7636239   0.06121919
pop            1.8228918           0.24128153      2.7114356   0.42041857
reggae         1.5043196           0.23902528      2.8492920   0.96777680
rock           2.1675093           0.24812642      1.3485636   0.96348978
              zero_crossing_rate_mean zero_crossing_rate_var harmony_mean
...
reggae      0.10947593  0.1366489
rock        0.09973316  0.1423806

Residual Deviance: 11084.3
AIC: 12074.3
```

Рисунок 2 – Summary модели в R

Построение модели KNN

Код для построения модели KNN на Python представлен в листинге 5.

Листинг 5 – KNN модель в Python

```
from sklearn.neighbors import KNeighborsClassifier

# Обучение модели
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)
```

Код для построения модели KNN на R представлен в листинге 6.

Листинг 6 – KNN модель в R

```
k <- 5
knn_pred <- knn(
  train = X_train_scaled,
  test = X_test_scaled,
  cl = y_train,
  k = k,
  prob = TRUE
)

# Получение вероятностей для KNN
knn_prob <- attr(knn_pred, "prob")
```

Оценка качества классификации

Код для оценки качества моделей на Python представлен в листинге 7.

Листинг 7 – ROC-кривые в Python

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
roc_auc_score, roc_curve, auc
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt
import numpy as np
y_test_bin = label_binarize(y_test, classes=np.arange(10))
print("Logistic Regression:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_logreg):.2f}")
print(f"Precision (macro): {precision_score(y_test, y_pred_logreg,
average='macro'):.2f}")
print(f"Recall (macro): {recall_score(y_test, y_pred_logreg,
average='macro'):.2f}")
y_pred_prob_logreg = logreg.predict_proba(X_test_scaled)
fpr_logreg = dict()
tpr_logreg = dict()
roc_auc_logreg = dict()
for i in range(10):
    fpr_logreg[i], tpr_logreg[i], _ = roc_curve(y_test_bin[:, i],
y_pred_prob_logreg[:, i])
    roc_auc_logreg[i] = auc(fpr_logreg[i], tpr_logreg[i])
plt.figure(figsize=(12, 6))
for i in range(10):
    plt.plot(fpr_logreg[i], tpr_logreg[i], lw=2, label=f'Класс {i} (AUC =
{roc_auc_logreg[i]:.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.title('Logistic Regression ROC (многоклассовая)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.show()
print("\nKNN:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.2f}")
print(f"Precision (macro): {precision_score(y_test, y_pred_knn,
average='macro'):.2f}")
print(f"Recall (macro): {recall_score(y_test, y_pred_knn, average='macro'):.2f}")
y_pred_prob_knn = knn.predict_proba(X_test_scaled)
fpr_knn = dict()
tpr_knn = dict()
roc_auc_knn = dict()
for i in range(10):
    fpr_knn[i], tpr_knn[i], _ = roc_curve(y_test_bin[:, i], y_pred_prob_knn[:,
i])
    roc_auc_knn[i] = auc(fpr_knn[i], tpr_knn[i])
plt.figure(figsize=(12, 6))
for i in range(10):
    plt.plot(fpr_knn[i], tpr_knn[i], lw=2, label=f'Класс {i} (AUC =
{roc_auc_knn[i]:.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.title('KNN ROC (многоклассовая)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.show()
```


Код для оценки качества моделей на R представлен в листинге 8.

Листинг 8 – ROC-кривые в R

```
# Подготовка данных для ROC-кривых
roc_data <- data.frame()
for (i in levels(y_test)) {
  y_test_bin <- ifelse(y_test == i, 1, 0)
  roc_obj <- roc(y_test_bin, y_pred_prob_logreg[, i], quiet = TRUE)

  # Добавляем данные для текущего класса
  roc_data <- rbind(roc_data, data.frame(
    Class = i,
    FPR = 1 - roc_obj$specificities,
    TPR = roc_obj$sensitivities
  ))
}

# Визуализация ROC-кривых
ggplot(roc_data, aes(x = FPR, y = TPR, color = Class)) +
  geom_line(size = 1) + # Линии ROC-кривых
  geom_abline(linetype = "dashed", color = "gray") + # Случайная модель
  labs(
    title = "Logistic Regression ROC (One-vs-Rest)",
    x = "False Positive Rate (1 - Specificity)",
    y = "True Positive Rate (Sensitivity)"
  ) +
  theme_minimal()

# Подготовка данных для ROC-кривых
roc_data_knn <- data.frame()
for (i in levels(y_test)) {
  y_test_bin <- ifelse(y_test == i, 1, 0)
  knn_prob_bin <- ifelse(knn_pred == i, knn_prob, 1 - knn_prob)
  roc_obj <- roc(y_test_bin, knn_prob_bin, quiet = TRUE)

  # Добавляем данные для текущего класса
  roc_data_knn <- rbind(roc_data_knn, data.frame(
    Class = i,
    FPR = 1 - roc_obj$specificities,
    TPR = roc_obj$sensitivities
  ))
}

# Визуализация ROC-кривых
ggplot(roc_data_knn, aes(x = FPR, y = TPR, color = Class)) +
  geom_line(size = 1) + # Линии ROC-кривых
  geom_abline(linetype = "dashed", color = "gray") + # Случайная модель
  labs(
    title = "KNN ROC (One-vs-Rest)",
    x = "False Positive Rate (1 - Specificity)",
    y = "True Positive Rate (Sensitivity)"
  ) +
  theme_minimal()
```

Визуализация ROC-кривых для логистической регрессии на Python представлена на рис. 3.

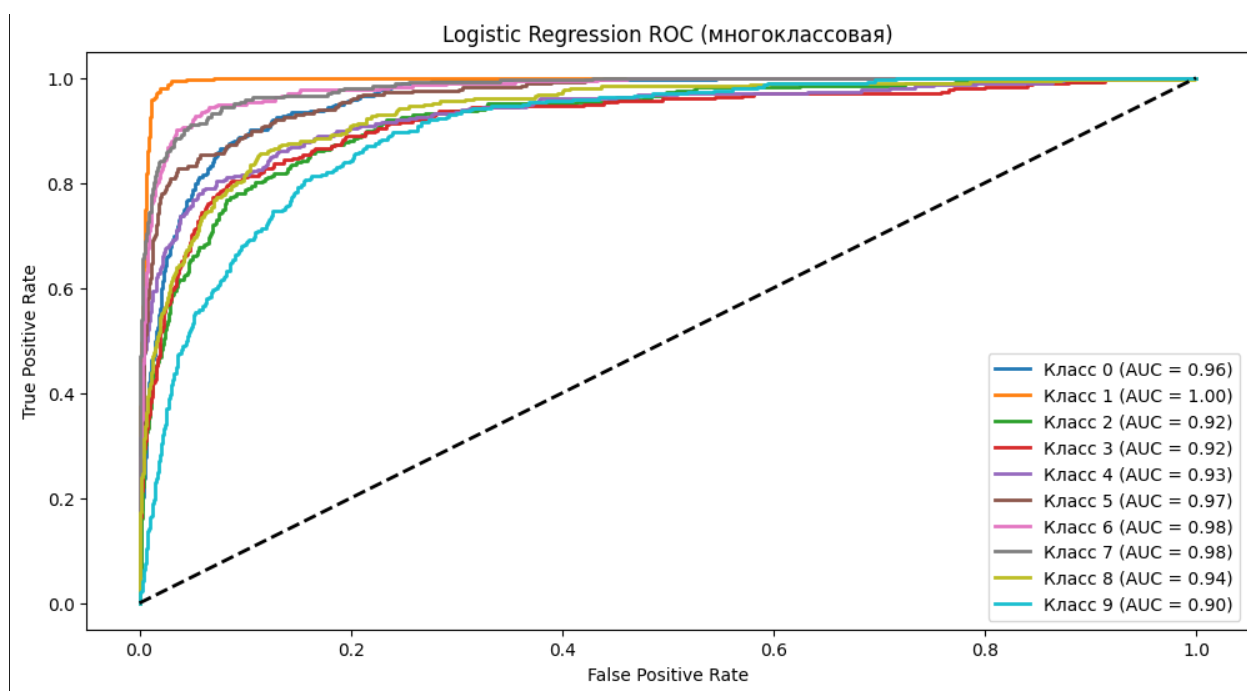


Рисунок 3– ROC-кривая логистической регрессии в Python

Визуализация ROC-кривых для KNN на Python представлена на рис. 4.

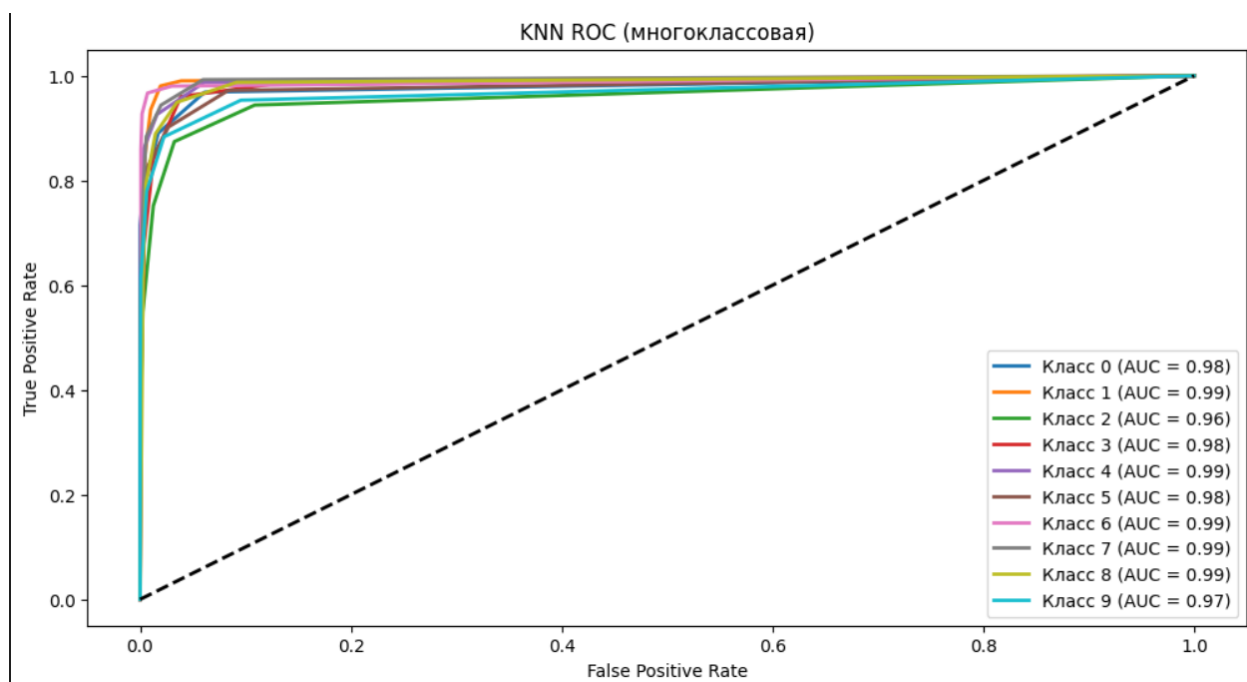


Рисунок 4– ROC-кривая KNN в Python

Визуализация ROC-кривых для логистической регрессии на R представлена на рис.

5.

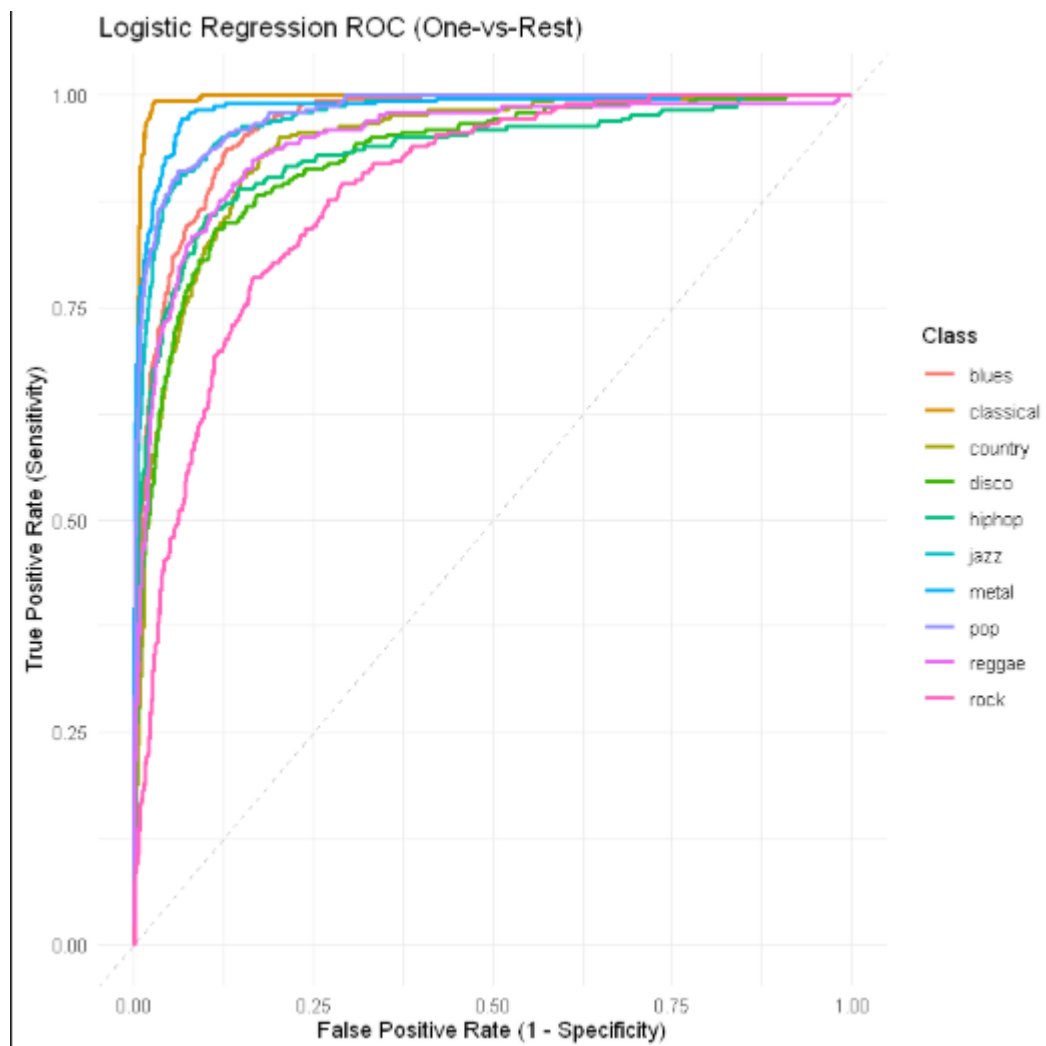


Рисунок 5— ROC-кривая логистической регрессии в R
Визуализация ROC-кривых для KNN на R представлена на рис. 6.

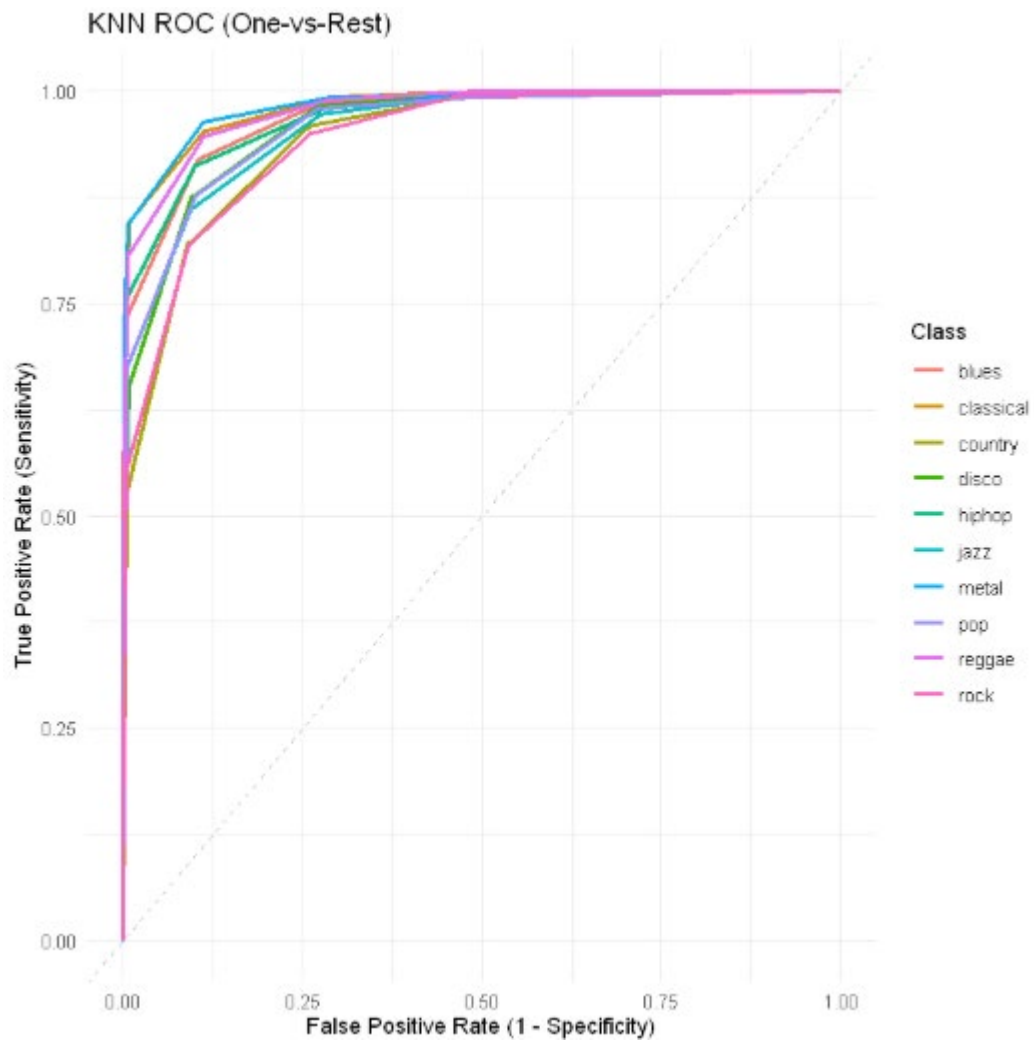


Рисунок 6 – ROC-кривая KNN в R

Интерпретация и сравнение результатов

Код для вывода сравнения моделей на Python представлен в листинге 9.

Листинг 9 – Сравнение моделей в Python

```
# Сравнение моделей
print("\nСравнение моделей:")
print(f"LogReg Macro-AUC: {np.mean(list(roc_auc_logreg.values())):.2f}")
print(f"KNN Macro-AUC: {np.mean(list(roc_auc_knn.values())):.2f}")
```

Код для вывода сравнения моделей на R представлен в листинге 10.

Листинг 10 – Сравнение моделей в R

```
# Сравнение моделей
conf_matrix_logreg <- confusionMatrix(factor(y_pred_logreg), factor(y_test))
conf_matrix_knn <- confusionMatrix(factor(knn_pred), factor(y_test))

results <- data.frame(
  Model = c("LogReg", "KNN"),
  Accuracy = c(conf_matrix_logreg$overall["Accuracy"],
conf_matrix_knn$overall["Accuracy"]),
  Macro_Precision = c(mean(conf_matrix_logreg$byClass[, "Precision"]),
mean(conf_matrix_knn$byClass[, "Precision"])),
  Macro_Recall = c(mean(conf_matrix_logreg$byClass[, "Recall"]),
mean(conf_matrix_knn$byClass[, "Recall"])),
  Macro_F1 = c(mean(conf_matrix_logreg$byClass[, "F1"]),
mean(conf_matrix_knn$byClass[, "F1"]))
)

print(results)
```

На основе полученных метрик можно сделать следующие выводы:

1. Логистическая регрессия показала схожие результаты в Python и R:
 - Accuracy ~0.72, Precision и Recall ~0.72.
 - Несмотря на простоту, модель демонстрирует умеренную точность, что может быть связано с линейной разделимостью данных.
 - Значение AUC = 0.95 (Python) указывает на высокую способность отделять классы.
2. KNN существенно превзошел логистическую регрессию:
 - В Python: Accuracy, Precision, Recall ~0.89, AUC = 0.98.
 - В R: Accuracy ~0.86, что может быть связано с отличиями в реализации алгоритма (например, выбором метрики расстояния или значения k).
 - Высокие показатели KNN объясняются его гибкостью в обработке нелинейных зависимостей, но требует тщательного подбора гиперпараметров.

ЗАКЛЮЧЕНИЕ

В рамках работы были построены и проанализированы модели классификации:

1. Логистическая регрессия подтвердила свою эффективность для линейно разделимых данных, обеспечив базовый уровень точности.
2. KNN показал наилучшие результаты, особенно в Python (Accuracy = 0.89, AUC = 0.98), что делает его предпочтительным для задач с комплексными зависимостями.
3. Наблюдаемые различия между Python и R могут быть связаны с особенностями реализации алгоритмов, методами предобработки данных или настройкой гиперпараметров.

Работа позволила освоить ключевые подходы к классификации, включая интерпретацию коэффициентов моделей, оценку метрик и визуализацию ROC-кривых. Полученные результаты подтверждают целесообразность использования KNN для аналогичных задач, но требуют дополнительного анализа для оптимизации параметров в R.